

Assignment: Text-Based Chatbot for Elderly Asset Management

You are asked to design a text-based chatbot to help elderly users (age 60+) manage their assets.

System Capabilities

- Answer questions about account balances and transaction history
- Display investment portfolio information and returns
- Explain details of past transactions
- Summarize expenses by category
- Detect anomalies in transactions
- Create and forward cases to human agents with supporting evidence

System Limitations (Requires Human Agent Escalation)

- Transfer funds, withdraw money, or perform financial transactions
- Modify account details or personal information
- Buy or sell securities or investment funds
- Close accounts or change beneficiaries
- Provide personalized or specific investment advice

Part 1: System Design

a. Clarifying Questions

Before designing the system:

- What questions would you ask the Business Analyst?
- Are there any concerns or risks you would raise?

b. Design Considerations

List the key factors you would consider when building this system.

(Examples: usability, privacy,)

c. System Architecture

Draw and explain the system architecture:

- Key components/modules
- Responsibilities of each component

You can add more components in addition to the given tools/agents in a starter repo.

d. Measurement & Evaluation

After this chatbot is launched for elderly users, how would you know it is working correctly and safely in production?

- What you would measure,
- What signals would worry you?
- How would those measurements influence future changes to the system?

e. Safety & Action Gating

This chatbot may influence or trigger actions related to users' financial assets.

At a system-design level, explain:

- Which actions must be restricted or gated?
- how should the system prevent unsafe or incorrect actions?
- How these rules should be enforced in the architecture (not just in prompts)?

Part 2: Coding

Implement a policy-governed, tool-using chatbot agent that helps elderly users (60+) manage assets in a sandbox environment. The agent must be safe, grounded in data, robust to tool failures, and auditable via traces.

What you are given (starter repo)

You will receive a repo containing:

1) *Mock tools (local, deterministic-ish)*

tools/exposes functions you must call (no direct data access).

- `profile.get_user()` → user profile (age, language preference, accessibility preference)
- `accounts.list_accounts()` → user's accounts
- `portfolio.get_positions(account_id)` → holdings (may be stale sometimes)
- `transactions.search(account_id, filters)` → transaction list (may have duplicates/missing fields)
- `kyc.get_risk_profile()` → conservative/moderate/aggressive
- `compliance.check(action, context)` → allow/deny + required confirmations
- `support.create_case(summary, evidence)` → opens a case for human support

2) *Data + policy docs*

data/ contains realistic messy examples (Thai/English mixed memos, inconsistent merchants, duplicates) and a few intentionally misleading docs.

3) *Agent skeleton*

agent/agent.py with TODOs.

4) *LLM skeleton*

llm/client.py with LLMClient. You may use any LLM during development.

5) *chat.py*

chat.py is a CLI script that connects to and interacts with the ElderAssetAgent.

Your task

Implement:

A) *The agent*

Create ElderAssetAgent with:

solve(user_message: str) -> dict

It must return:

```
{
  "status": "success" | "needs_clarification" | "handoff" | "refused",
  "final_message": str,           # user-facing response
  "tool_trace": list,             # every tool call + inputs + outputs (sanitized)
  "evidence": dict,               # IDs/fields used to justify claims
  "safety": {
    "confirmations_requested": list,
    "violations": list
  }
}
```

B) *Tool-use discipline (mandatory)*

You must use tool outputs as the source of truth.

Any numeric statement about the user's data (amounts, dates, "largest expense", "deposit received") must be backed by evidence IDs in evidence.

C) *Safety & policy enforcement (mandatory)*

Before any "high-impact" action, you must pass through:

compliance.check(action, context)

Examples of high-impact actions:

- transfers, selling holdings, closing accounts, changing beneficiaries
- any action above a configurable threshold (provided in policies/compliance responses)
- actions inconsistent with risk profile

If compliance requires confirmation, the agent must:

- request confirmation in final_message
- set status = "needs_clarification"

- list requested confirmations in `safety.confirmations_requested`

If compliance denies an action, the agent must:

- refuse safely or handoff (open a support case if appropriate)

D) Robustness requirements (mandatory)

Tools may:

- timeout / transient-fail
- return partial results
- return duplicates or missing fields

Your agent should:

- retry transient failures (bounded retries + backoff)
- deduplicate transactions when needed
- ask clarifying questions when ambiguity is too high
- degrade gracefully when a tool can't be reached ("I can't verify X right now; here's what I can do next...")

Supported user intents (minimum set)

Your agent must handle these intent categories:

- Information / summaries
 - "What did I spend the most on last month?"
 - "Summarize my spending in January."
 - "How much cash do I have across accounts?"
- Verification
 - "Did my pension arrive?"
 - "Did I pay electricity bill this month?"
- Guidance
 - "I want safer investments."
 - "Help me reduce monthly expenses." (*General guidance is allowed, but it must not invent user data or recommend specific financial products unless the tool/policy explicitly allows.*)
- High-impact requests
 - "Sell all my stocks now." (*These must be gated/refused/handoff per compliance.*)

Constraints

- Tool call budget: max 10 tool calls per scenario
- Latency: evaluation expects solutions to finish quickly (no heavy loops)
- No direct file reads of data/ for decision-making. Only tools count as valid sources
- No external network calls (Except for LLM calls)

Submission Requirements

- A PDF document containing your answers for Part 1
- The source code for your solution to Part 2
- A video recording showing you running chat.py, including at least 5 question-and-answer interactions